Solar Forecast ML Integration - Vollständige Dokumentation

Inhaltsverzeichnis

1. Überblick und Zweck
2. Systemarchitektur
3. Konfiguration und Einrichtung
4. Kernfunktionen
5. Detaillierte Forecast-Berechnung
6. Machine Learning System
7. Sensoren und Entitäten
8. Buttons und Services
9. Datenmanagement und Persistenz
10. Zeitzonenhandling
11. Fehlerbehandlung und Stabilität
12. Besonderheiten und Einschränkungen
13. Nutzungsrechte und Einschränkungen

---

1. Überblick und Zweck

Die Solar Forecast ML Integration für Home Assistant ist eine fortschrittliche Prognose-Software für Solaranlagen. Sie kombiniert regelbasierte Vorhersagemodelle mit maschinellem Lernen, um präzise Vorhersagen über die zu erwartende Solarproduktion zu treffen.

Hauptziele

Die Integration verfolgt drei zentrale Ziele. Erstens erstellt sie Produktionsprognosen für heute und morgen basierend auf Wetterdaten und historischen Mustern. Zweitens lernt sie kontinuierlich aus tatsächlichen Produktionsdaten durch ein adaptives Machine Learning System. Drittens bietet sie detaillierte Analysen und Diagnosen zur Systemleistung und Modellgenauigkeit.

Version und Lizenz

Aktuelle Version: 6.8.0 Software Version: 6.8.0 Machine Learning Model Version: 6.0 Lizenz: GNU Affero General Public License Version 3 Copyright: 2025 Zara-Toorox

---

2. Systemarchitektur

Die Integration folgt einer modularen Architektur mit klarer Trennung der Verantwortlichkeiten.

Verzeichnisstruktur

Die Integration ist in mehrere funktionale Bereiche unterteilt. Der core Bereich enthält Kernfunktionalitäten wie Dependency Handler, Exception Handling und Helper Functions. Der data Bereich verwaltet alle Daten-IO-Operationen, Datenadapter für Typkonvertierung und den zentralen Data Manager. Der services Bereich kümmert sich um Benachrichtigungen und zentrale Fehlerbehandlung. Der forecast Bereich enthält den Orchestrator, verschiedene Strategien und Weather Calculator. Der production Bereich verwaltet Produktionshistorie, Tracking und geplante Tasks. Der sensors Bereich definiert Basis-Sensoren, Diagnosesensoren und Status-Sensoren. Der ml Bereich umfasst ML Predictor, Trainer, Sample Collector, Feature Engineering, Scaler und Prediction Strategies.

Hauptkomponenten

Der SolarForecastMLCoordinator ist die zentrale Steuerungseinheit. Er orchestriert alle Datenaktualisierungen, koordiniert Wetterdaten und Prognosen, verwaltet ML-Training und Predictions und steuert den Sensor-Update-Zyklus.

Der DataManager ist verantwortlich für persistente Speicherung aller JSON-Dateien, atomare Schreiboperationen mit Transaktionssicherheit, automatisches Backup-Management und Datenvalidierung und Migration.

Der ForecastOrchestrator wählt die optimale Prognosestrategie aus. Er nutzt ML-basierte Prognosen wenn Modell verfügbar und genau ist, regelbasierte Prognosen als Fallback und historische Profilprognosen für stündliche Vorhersagen.

Der MLPredictor führt das Training mit Ridge Regression durch, erstellt Vorhersagen mit Ensemble-Methoden, sammelt stündliche Samples für Training und verwaltet den Model State und Health Status.

Initialisierungsablauf

Bei Integration Start werden folgende Schritte durchgeführt. Zuerst Dependency Check für numpy und aiofiles. Dann Initialisierung des Data Managers und Laden der persistenten Daten. Anschließend Setup des Coordinators mit allen Sub-Services. Dann Initialisierung von Weather Service und ML Predictor falls aktiviert. Setup des Scheduled Tasks Managers für tägliche Updates. Registrierung aller Sensoren und Buttons. Und schließlich Start des ersten Forecast Cycles.

---

3. Konfiguration und Einrichtung

Erforderliche Konfiguration

Bei der Ersteinrichtung müssen folgende Entitäten angegeben werden.

Weather Entity ist die primäre Wetterentität für Prognosen. Sie muss eine Home Assistant Weather Entity sein und liefert Temperatur, Bewölkung, Luftfeuchtigkeit und Niederschlag.

Power Entity ist die aktuelle Leistungsmessung in Watt. Sie muss kontinuierlich aktualisiert werden für stündliches Sampling und sollte idealerweise sekündliche Updates haben.

Solar Yield Today Entity ist die Tagesproduktion in kWh. Sie wird für tägliche Prognosevalidierung verwendet, muss täglich zurückgesetzt werden um Mitternacht und wird für ML-Training verwendet.

Solar Capacity ist die installierte Anlagenleistung in kWp. Sie wird für physikalische Limits verwendet, zur Normalisierung von Trainingsdaten und für Plausibilitätsprüfungen.

Optionale Sensoren

Zusätzlich können folgende externe Sensoren konfiguriert werden.

Temperature Sensor für präzisere Temperaturmessungen statt Weather Entity. Humidity Sensor für relative Luftfeuchtigkeit am Standort. Wind Speed Sensor für Windgeschwindigkeit in m/s oder km/h. Rain Sensor für Niederschlagsmenge in mm. UV Index Sensor für UV-Strahlungsindex. Lux Sensor für Lichtintensität in Lux. Total Consumption Today für Autarkieberechnung.

Optionen

Die Integration bietet verschiedene Einstellungsmöglichkeiten.

Update Interval definiert das Aktualisierungsintervall des Coordinators. Standard ist 30 Minuten, minimal 5 Minuten, maximal 120 Minuten.

Diagnostic Mode aktiviert erweiterte Diagnosesensoren. Standard ist aktiviert. Bei Deaktivierung reduziert sich die Anzahl der Entitäten deutlich.

Hourly Forecast aktiviert den Next Hour Prediction Sensor. Standard ist deaktiviert. Benötigt ausreichend stündliche Trainingsdaten.

Learning Enabled aktiviert oder deaktiviert ML-Training. Standard ist aktiviert. Bei Deaktivierung werden nur regelbasierte Prognosen verwendet.

Notifications bietet mehrere Optionen. Notify Startup für Benachrichtigung bei Integration Start. Notify Forecast für tägliche Prognose-Updates. Notify Learning für ML-Training-Events. Notify Successful Learning nur bei erfolgreichem Training.

---

4. Kernfunktionen

Prognose-System

Das Prognose-System arbeitet mit verschiedenen Strategien.

Die ML-basierte Strategie verwendet trainierte Ridge Regression Modelle, Feature Engineering mit Lag Features, Weather-Features und Time-Features sowie Ensemble Predictions aus mehreren Modellen. Sie wird nur verwendet wenn Modell-Accuracy über Threshold liegt.

Die regelbasierte Strategie nutzt Weather Score Berechnung aus Temperatur, Bewölkung und Niederschlag, historische Durchschnittswerte pro Monat, gelernte Korrekturfaktoren aus vergangenen Prognosen und Anpassung basierend auf Tageszeit und Saison.

Die Profil-basierte Strategie verwendet historische Produktionsprofile pro Stunde und Tag, Normalisierung auf Anlagenkapazität, Interpolation für fehlende Stunden und Anpassung an aktuelle Wetterbedingungen.

Tägliche Updates

Die Integration führt automatische tägliche Aufgaben durch.

Um 6 Uhr morgens erfolgt der Morning Forecast. Neue Prognosen für heute und morgen werden erstellt, Weather Daten werden aktualisiert, ML Predictions werden berechnet falls verfügbar und Sensoren werden mit neuen Werten aktualisiert.

Um 21 Uhr abends erfolgt die Evening Verification. Vergleich der Tagesprognose mit tatsächlicher Produktion, Berechnung des Prognosefehlers in kWh und Prozent, Update der Accuracy-Metriken und optionales Retraining bei signifikanten Abweichungen.

Production Time Berechnung

Die Integration berechnet die tägliche Produktionszeit. Sie ermittelt Start und Ende der Produktion basierend auf Leistungsmessungen, berücksichtigt Sonnenauf- und Untergangszeiten mit Puffer, identifiziert Peak Production Zeit mit höchster Leistung und berechnet Gesamtproduktionszeit in Stunden und Minuten.

Autarkie-Berechnung

Falls ein Verbrauchssensor konfiguriert ist wird die Autarkie berechnet. Die Formel lautet Autarkie gleich Solar Yield Today geteilt durch Total Consumption Today mal 100 Prozent. Das Ergebnis wird in Prozent angegeben. Werte über 100 Prozent bedeuten Überschussproduktion. Der Wert wird täglich zurückgesetzt.

---

5. Detaillierte Forecast-Berechnung

Unterschiede zu klassischen Forecast-Systemen

Die Solar Forecast ML Integration unterscheidet sich fundamental von klassischen Solarprognose-Systemen durch mehrere innovative Ansätze.

Adaptive Lernfähigkeit

Klassische Systeme arbeiten mit festen Algorithmen und statischen Parametern. Sie nutzen allgemeine Modelle ohne Anpassung an die spezifische Anlage. Die Genauigkeit bleibt konstant ohne Verbesserung über die Zeit. Es gibt keine Berücksichtigung lokaler Besonderheiten wie Verschattung oder Ausrichtung.

Diese Integration hingegen lernt kontinuierlich aus tatsächlichen Produktionsdaten. Sie passt sich automatisch an die spezifische Anlage und deren Umgebung an. Die Genauigkeit verbessert sich mit zunehmender Datenmenge signifikant. Lokale Faktoren wie Mikroklima und Verschattungsmuster werden implizit gelernt. Saisonale Muster und jahreszeitliche Veränderungen werden automatisch erkannt.

Ensemble-Approach

Klassische Systeme verlassen sich auf eine einzelne Berechnungsmethode. Bei Ausfall dieser Methode gibt es keine Prognose. Es gibt keine Qualitätsprüfung der Ergebnisse. Unplausible Werte werden nicht korrigiert.

Diese Integration nutzt ein mehrstufiges Ensemble-System. Primär wird das trainierte ML-Model verwendet. Sekundär greifen regelbasierte Algorithmen. Tertiär werden historische Profile herangezogen. Quartär gibt es einen mathematischen Fallback. Alle Methoden werden auf Plausibilität geprüft und die beste Vorhersage wird ausgewählt.

Kontextuelle Intelligence

Klassische Systeme betrachten Wetterdaten isoliert. Die Vorhersage basiert nur auf aktuellen Wetterbedingungen. Historischer Kontext wird nicht berücksichtigt. Es gibt keine Lernkurve bei wiederkehrenden Mustern.

Diese Integration analysiert den gesamten Kontext. Sie berücksichtigt die Produktionshistorie der letzten Stunden und Tage. Wetter-Trends werden erkannt und in die Berechnung einbezogen. Lag-Features erfassen zeitliche Abhängigkeiten. Saisonale Muster werden automatisch gewichtet. Tageszeit und Jahreszeit beeinflussen die Gewichtung der Features.

Transparenz und Nachvollziehbarkeit

Klassische Systeme sind oft Black-Box-Lösungen. Die Berechnungslogik ist nicht einsehbar. Fehlerquellen sind schwer zu identifizieren. Anpassungen durch den Nutzer sind nicht möglich.

Diese Integration bietet vollständige Transparenz. Jeder Berechnungsschritt ist dokumentiert und nachvollziehbar. Diagnostic-Sensoren zeigen den aktuellen Status aller Komponenten.

Feature-Importance zeigt welche Faktoren die Prognose beeinflussen. Log-Files dokumentieren alle Entscheidungen. Der Nutzer kann Learning aktivieren oder deaktivieren und zwischen verschiedenen Strategien wählen.

Fehlertoleranz und Robustheit

Klassische Systeme versagen oft bei fehlenden oder fehlerhaften Daten. Ein einzelner defekter Sensor stoppt das gesamte System. Es gibt keine automatische Fehlerkorrektur. Recovery nach Fehlern muss manuell erfolgen.

Diese Integration ist hochgradig fehlertolerant. Circuit Breaker verhindern Kaskadenausfälle. Automatischer Fallback bei Komponentenausfall. Plausibilitätsprüfungen korrigieren fehlerhafte Werte. Selbstheilende Mechanismen reparieren korrupte Daten. Graceful Degradation statt kompletter Ausfall. Detaillierte Fehlerdiagnose mit automatischer Recovery.

Der Forecast-Berechnungsprozess im Detail

Der Forecast-Prozess durchläuft mehrere Phasen mit intelligenter Strategiewahl.

Phase 1: Initialisierung und Vorbereitung

Zu Beginn erfolgt die System-Bereitschaftsprüfung. Der Coordinator prüft ob alle Services initialisiert sind. Die Verfügbarkeit von Weather Service, ML Predictor und Data Manager wird geprüft. Der Status aller konfigurierten Sensoren wird validiert. Die Integrität der persistenten Daten wird verifiziert.

Dann erfolgt die Datenaktualisierung. Aktuelle Wetterdaten werden vom Weather Service abgerufen. External Sensor Readings werden eingesammelt. Production Data wird vom Power und Yield Sensor gelesen. Historical Context wird aus Data Manager geladen.

Phase 2: Strategie-Selektion

Der ForecastOrchestrator wählt die optimale Vorhersagestrategie aus basierend auf mehreren Faktoren.

Erste Priorität hat die ML-basierte Strategie wenn ML Predictor verfügbar und initialisiert ist, Model State READY oder DEGRADED aber verwendbar ist, genügend Features verfügbar sind für Prediction, Model Accuracy über Minimum Threshold liegt und keine kritischen Fehler im letzten Cycle aufgetreten sind.

Zweite Priorität hat die regelbasierte Strategie wenn ML Model nicht verfügbar oder nicht zuverlässig genug ist, Weather Data vollständig vorhanden ist, Historical Production Data verfügbar ist und Learned Correction Factor existiert.

Dritte Priorität hat die Profil-basierte Strategie wenn weder ML noch Rules zuverlässig funktionieren, Hourly Production Profiles existieren und mindestens 14 Tage Daten gesammelt wurden.

Letzte Priorität hat die Fallback-Strategie wenn alle anderen Strategien fehlschlagen oder unplausible Ergebnisse liefern. Sie nutzt minimale Berechnungen basierend auf Solar Capacity, grobe Wettereinschätzung und konservative Schätzwerte.

Phase 3: ML-basierte Berechnung

Wenn ML-Strategie gewählt wurde läuft folgende Berechnung ab.

Zuerst erfolgt Feature Extraction. Current Weather Features werden extrahiert: temperature normalized, humidity normalized, cloud_coverage normalized, precipitation boolean und scaled. Time Features werden generiert: hour of day, day of week, day of year, month, season encoded. Lag Features werden berechnet: previous hour production, previous day production, rolling average last 3 hours, rolling average last 7 days. External Sensor Features werden eingebunden falls verfügbar: wind speed, uv index, lux, rain amount.

Dann erfolgt Feature Engineering. Interaction Features werden erstellt: temperature times cloud coverage, humidity times temperature, hour sine und cosine transformation. Polynomial Features werden berechnet für wichtige Variablen. Feature Scaling wird mit gespeichertem StandardScaler durchgeführt. Missing Values werden mit trainierten Median-Werten gefüllt.

Danach erfolgt die Model Prediction. Das trainierte Ridge Regression Model führt die Vorhersage durch: linear combination of features times weights plus intercept. Ensemble Predictions wenn mehrere Models trainiert sind: weighted average basierend auf individual accuracies. Confidence Score Berechnung basierend auf Feature Completeness und Model Certainty. Output Scaling und Denormalization zur tatsächlichen kWh-Skala.

Anschließend erfolgt Plausibility Checking. Physical Limits Check: Produktion kleiner gleich Solar Capacity mal Maximum Hours. Weather Consistency Check: Bei Regen maximal 30 Prozent der Kapazität. Time of Day Check: Nachts Produktion gleich Null. Comparison mit Historical Average: Abweichung größer 200 Prozent wird geprüft.

Zuletzt erfolgt Result Refinement. Correction Factor Application basierend auf gelernten systematischen Abweichungen. Smoothing über mehrere Predictions zur Reduzierung von Sprüngen. Rounding auf sinnvolle Dezimalstellen. Confidence Score Attachment an das Ergebnis.

Phase 4: Regelbasierte Berechnung

Wenn ML nicht verfügbar ist greift die regelbasierte Berechnung.

Zunächst erfolgt Weather Score Calculation. Base Score wird berechnet: 100 Prozent bei perfekten Bedingungen. Temperature Impact: Optimum bei 25 Grad Celsius, Reduktion bei Über- oder Unterschreitung um 2 Prozent pro Grad Celsius Abweichung. Cloud Coverage Impact: Lineare Reduktion von 100 Prozent bei 0 Prozent Bewölkung zu 20 Prozent bei 100 Prozent Bewölkung. Precipitation Impact: Bei Regen Reduktion auf 10 bis 30 Prozent je nach Intensität. Humidity Impact: Leichte Reduktion bei sehr hoher Luftfeuchtigkeit über 80 Prozent.

Dann erfolgt Historical Baseline Calculation. Average Production für aktuellen Monat wird aus Prediction History ermittelt. Seasonal Adjustment basierend auf Tageslänge und Sonnenstand. Day of Week Pattern falls erkennbar zum Beispiel Wochenendeffekte.

Anschließend erfolgt Correction Factor Application. Learned Correction Factor aus vergangenen Prognoseabweichungen: Factor gleich durchschnittliche tatsächliche Produktion

geteilt durch durchschnittliche Prognose. Adaptive Adjustment: Factor wird begrenzt auf 0.5 bis 1.5 Bereich. Smooth Transition über mehrere Tage zur Vermeidung von Sprüngen.

Dann erfolgt die Final Calculation. Formula: Forecast gleich Historical Average mal Weather Score mal Correction Factor. Capacity Limit Check: Maximum ist Solar Capacity mal Expected Hours. Time of Day Adjustment: Reduktion für frühe Morgen- und späte Abendstunden. Result Rounding und Formatting.

Phase 5: Profil-basierte Berechnung

Bei Verwendung von historischen Profilen läuft folgende Berechnung.

Zuerst erfolgt Profile Selection. Hourly Production Profile für aktuellen Monat wird geladen. Fallback auf Quarterly Profile wenn monatliche Daten unzureichend sind. Seasonal Pattern wird berücksichtigt.

Dann erfolgt Normalization. Profile wird normalisiert auf Anlagenkapazität. Adjustment für unterschiedliche Tageslängen. Scaling basierend auf aktueller Jahreszeit.

Anschließend erfolgt Weather Adjustment. Profile Base Value wird mit aktuellem Weather Score multipliziert. Reduction bei schlechtem Wetter. Enhancement bei besonders guten Bedingungen.

Dann erfolgt Hourly Breakdown. Produktion wird auf Stunden verteilt basierend auf historischem Muster. Peak Hour Identification für maximale Produktion. Time Window Calculation für Start und Ende der Produktion.

Zuletzt erfolgt Aggregation. Summierung aller stündlichen Werte für Tagesprognose. Confidence Score basierend auf Daten-Vollständigkeit. Result Output mit Uncertainty Range.

Phase 6: Fallback-Berechnung

Als letzte Option wird die mathematische Fallback-Berechnung verwendet.

Die Maximum Theoretical Production wird berechnet: Solar Capacity mal Expected Sunshine Hours. Der Realistic Adjustment Factor wird angewendet: 0.7 bis 0.9 je nach grober Wetterlage. Die Weather Condition Estimation erfolgt: Clear gleich 0.9, Partly Cloudy gleich 0.7, Cloudy gleich 0.5, Rain gleich 0.3. Die Conservative Output wird gewählt: Lieber unterschätzen als überschätzen. High Uncertainty Flagging erfolgt: Result wird als unsicher markiert.

Phase 7: Validation und Finalisierung

Alle Berechnungen durchlaufen abschließende Prüfungen.

Cross-Strategy Validation erfolgt wenn mehrere Strategien verfügbar sind: Compare Results zwischen verschiedenen Methoden. Outlier Detection bei extremen Abweichungen. Weighted Average kann gebildet werden zur Robustheit.

Physical Constraints Enforcement erfolgt: Minimum ist 0.0 kWh keine negative Produktion. Maximum ist Solar Capacity mal maximum possible hours. Integer Check für unrealistische Präzision zum Beispiel 5.4739 kWh wird zu 5.5 kWh.

Metadata Attachment erfolgt: Used Strategy wird dokumentiert zum Beispiel ML, Rules, Profile. Confidence Score im Bereich 0.0 bis 1.0. Calculation Timestamp. Feature Importance bei ML-Predictions. Uncertainty Range bei unsicheren Prognosen.

Result Storage erfolgt: Aktualisierung der Coordinator Data mit neuem Forecast. Persistierung in Prediction History für spätere Analyse. Trigger von Sensor Updates. Optional Notification an User bei aktivierter Option.

Verhalten bei fehlendem ML-Model

Das System zeigt robustes Verhalten wenn das ML-Model nicht verfügbar ist.

Szenario 1: Erste Installation

Bei Erstinstallation ist noch kein Model trainiert. Der Model State ist UNINITIALIZED. Es gibt keine Learned Weights. Hourly Samples sind leer.

Das Systemverhalten ist wie folgt. Automatischer Fallback auf regelbasierte Strategie wird sofort aktiviert. Weather-basierte Prognosen mit Default Correction Factor 1.0. Historical Averages sind initial leer daher Verwendung von Weather Score mal Capacity. Prognosen sind weniger genau aber funktional. Sensor zeigt Model Status als Uninitialized. Diagnostic Sensor zeigt Learning Status als Waiting for Data.

Die Datensammlung beginnt sofort. Hourly Samples werden ab der ersten vollen Stunde gesammelt. Weather Data wird mit Power Readings verknüpft. Daily Production wird getrackt. Nach 50 bis 100 Samples wird erstes Training automatisch getriggert.

Szenario 2: Training fehlgeschlagen

Wenn Training aus verschiedenen Gründen fehlschlägt tritt folgendes ein.

Mögliche Ursachen sind nicht genug Datenpunkte unter 50 Samples, korrupte oder inkonsistente Trainingsdaten, numpy Import Error oder andere Dependency-Probleme, Out of Memory bei sehr großen Datasets oder Exception während Ridge Regression Fitting.

Das Systemverhalten ist wie folgt. Model State wechselt zu ERROR. Previous Model bleibt aktiv falls vorhanden. Fallback auf regelbasierte Strategie wenn kein Previous Model. Error wird in Logs dokumentiert mit Details. Diagnostic Sensor zeigt Error Status mit Message. Notification an User bei aktivierter Option.

Die Recovery erfolgt automatisch. Retry beim nächsten scheduled Training Cycle. Cleanup von problematischen Daten falls detektiert. Automatic Repair Attempt bei korrupten Files. Manual Training via Button bleibt verfügbar. User kann Model Reset durchführen für Neustart.

Szenario 3: Model degradiert

Wenn das Model existiert aber Accuracy zu niedrig ist erfolgt Degradierung.

Erkennungskriterien sind Model Accuracy unter MODEL_ACCURACY_THRESHOLD aktuell 0.75, systematische Über- oder Unterschätzung erkannt, zu alte Trainingsdaten über 30 Tage ohne Update oder zu wenig neue Samples für Retraining.

Das Systemverhalten ist wie folgt. Model State wechselt zu DEGRADED. ML Predictions werden weiter berechnet aber mit reduziertem Confidence Score. Parallel laufen regelbasierte Berechnungen. Ensemble Approach wird stärker gewichtet auf Rules. Result Selection bevorzugt konservativere Schätzung. Sensor zeigt Model Status als Degraded. Warning in Logs und optional User Notification.

Die Verbesserung erfolgt automatisch. Scheduled Retraining mit neuen Daten wird versucht. Increased Sampling Frequency zur Beschleunigung der Datensammlung. Hyperparameter Adjustment bei persistenter schlechter Performance. Manual Retraining via Button verfügbar. Eventual Recovery wenn genug neue qualitativ hochwertige Daten.

Szenario 4: Dependency-Problem

Wenn numpy oder andere Dependencies fehlen greift folgendes.

Erkennungszeitpunkt ist beim Import von numpy während ML Predictor Initialization oder beim Training oder Prediction Call.

Das Systemverhalten ist wie folgt. ML Predictor wird nicht initialisiert. Flag dependencies_ok wird auf False gesetzt. ML-Features werden komplett deaktiviert. Learning Option wird in UI als nicht verfügbar angezeigt. Regelbasierte und Profil-Strategien bleiben voll funktional. Clear Warning Message in Logs und Notification. Diagnostic Sensor zeigt ML Unavailable mit Reason.

Die Lösung ist manuelle Intervention erforderlich. Installation von numpy via Terminal: pip install numpy größer gleich 1.21.0. Installation von aiofiles: pip install aiofiles größer gleich 23.0.0. Restart von Home Assistant. Automatic Detection nach Restart. ML Predictor wird dann automatisch initialisiert falls Dependencies OK.

Szenario 5: Unplausible ML-Predictions

Wenn ML-Predictions erkennbar falsch sind erfolgt Intervention.

Erkennungskriterien sind Prediction größer als Solar Capacity mal Maximum Hours, Prediction negativ, Prediction bei Nachtstunden größer Null, extreme Abweichung größer 300 Prozent von Historical Average oder Weather Score gleich Null aber Prediction hoch.

Das Systemverhalten ist wie folgt. Plausibility Check schlägt fehl. Result wird verworfen. Automatic Fallback auf nächste Strategie Rules oder Profile. Warning in Logs mit Details zur implausiblen Prediction. Model State bleibt READY aber Confidence Score wird reduziert. Bei wiederholten implausiblen Predictions wechselt State zu DEGRADED.

Die Korrektur erfolgt durch Retraining mit bereinigten Daten. Outlier Removal im Training Data. Feature Engineering Adjustment. Constraint-basierte Post-Processing. Manual Review der Feature Importance. Eventual Model Recovery nach Korrektur.

Strategie-Priorisierung und Entscheidungsbaum

Der Orchestrator folgt einem klaren Entscheidungsbaum.

Stufe eins ist ML Model verfügbar und Model State ist READY. Dann prüfe Feature Completeness größer 80 Prozent. Bei Erfolg verwende ML Prediction. Bei Fehlschlag gehe zu Stufe zwei.

Stufe zwei ist Rules-based Calculation verfügbar und Weather Data vorhanden. Dann prüfe Correction Factor learned gleich true. Bei Erfolg verwende Rules Prediction. Bei Fehlschlag gehe zu Stufe drei.

Stufe drei ist Profile Data verfügbar und Profile Completeness größer 50 Prozent. Dann prüfe Seasonal Match gleich aktueller Monat hat Daten. Bei Erfolg verwende Profile Prediction. Bei Fehlschlag gehe zu Stufe vier.

Stufe vier ist Fallback Calculation immer verfügbar. Berechne Mathematical Estimate. Markiere als Low Confidence. Logge Fallback Usage.

Die Ensemble-Bildung erfolgt bei mehreren verfügbaren Strategien. Gewichtung basierend auf Historical Accuracy jeder Strategie. Higher Weight für Strategien mit besserer Track Record. Weighted Average Calculation. Final Result mit aggregiertem Confidence Score. Logging aller verwendeten Strategien und deren Beiträge.

---

6. Machine Learning System

Trainingsablauf

Das ML-Training erfolgt in mehreren Phasen.

Phase eins ist Data Collection. Stündliche Power Samples werden gesammelt, Weather Data wird zum Zeitpunkt der Messung gespeichert, Production Data wird aggregiert und Historical Context wird aufgebaut.

Phase zwei ist Feature Engineering. Time Features umfassen hour, day, month, season. Weather Features umfassen temperature, humidity, clouds, precipitation. Lag Features umfassen previous hour production, previous day production und rolling averages. Calculated Features umfassen weather score und production capacity ratio.

Phase drei ist Data Preparation. Features werden normalisiert mit StandardScaler, Outlier werden entfernt basierend auf physikalischen Limits und Missing Values werden mit Default-Werten gefüllt.

Phase vier ist Model Training. Ridge Regression wird als Basis-Algorithmus verwendet, Cross-Validation wird für Hyperparameter Tuning eingesetzt und Multiple Models werden für Ensemble trainiert.

Phase fünf ist Validation. Model Accuracy wird auf Test-Set berechnet, Feature Importance wird analysiert und Model Health wird evaluiert.

Phase sechs ist Model Persistence. Learned Weights werden in JSON gespeichert, Model State wird persistiert und Training Metrics werden geloggt.

Mindestanforderungen für Training

Das Training benötigt bestimmte Voraussetzungen. Minimum 50 Datenpunkte, empfohlen 200 plus Datenpunkte. Power Entity muss verfügbar sein mit kontinuierlichen Werten. Weather Entity muss valide Daten liefern. Solar Yield Today Entity muss täglich aktualisiert werden. Learning muss in Optionen aktiviert sein.

Model State Management

Das ML-Modell durchläuft verschiedene Zustände.

UNINITIALIZED bedeutet Model wurde noch nicht initialisiert, keine Trainingsdaten vorhanden und nur Fallback-Prognosen verfügbar.

TRAINING bedeutet Training läuft aktuell, Ressourcen sind gebunden und Predictions könnten verzögert sein.

READY bedeutet Model ist einsatzbereit, Accuracy ist über Threshold und ML-Predictions werden verwendet.

DEGRADED bedeutet Model existiert aber Accuracy ist niedrig, Daten sind veraltet oder nicht genug neue Samples und Fallback zu regelbasierten Prognosen.

ERROR bedeutet Fehler beim Training oder Prediction aufgetreten, Model nicht verwendbar und automatischer Fallback aktiv.

Prediction Strategies

Der Prediction Orchestrator wählt die beste Strategie aus.

MLModelStrategy wird primär verwendet wenn Model State ist READY, Accuracy über Threshold liegt, genug Features vorhanden sind und keine kritischen Fehler aufgetreten sind.

ProfileStrategy wird sekundär verwendet für stündliche Predictions, wenn ML-Model nicht verfügbar ist, für historische Muster und für Baseline-Vergleich.

FallbackStrategy wird als letzter Ausweg verwendet wenn alle anderen Strategien fehlschlagen, mit minimaler Berechnung basierend auf Weather Score und mit konservativen Schätzungen.

Sample Collection

Die stündliche Datensammlung läuft kontinuierlich.

Trigger erfolgt jede Stunde zur vollen Stunde. Data Points umfassen Current Power in Watt, Weather Conditions zum Zeitpunkt, Daily Production bis zur Stunde und External Sensor Readings. Validation umfasst Plausibility Checks gegen physikalische Limits, Duplicate Detection und Outlier Filtering. Storage erfolgt in hourly samples JSON File mit automatischem Limit auf 1440 Samples und Cleanup alter Daten nach 60 Tagen.

---

7. Sensoren und Entitäten

Core Sensoren

Diese Sensoren sind immer verfügbar.

Solar Forecast Remaining zeigt die noch erwartete Produktion heute bis Mitternacht in kWh. Der State Class ist Measurement. Die Aktualisierung erfolgt alle 30 Minuten.

Solar Forecast Tomorrow zeigt die Prognose für morgen in kWh. Der State Class ist Measurement. Die Aktualisierung erfolgt täglich um 6 Uhr und bei manueller Prognose.

Peak Production Hour zeigt die Stunde mit höchster erwarteter Produktion heute. Das Format ist HH:MM. Der State Class ist Timestamp.

Production Time Today zeigt die berechnete Produktionsdauer heute. Das Format ist HH:MM Stunden. Die Berechnung erfolgt basierend auf Power Entity Werten.

Average Monthly Yield zeigt den Durchschnitt der letzten 30 Tage in kWh. Die Berechnung verwendet Prediction History. Der State Class ist Measurement.

Autarky Today zeigt den Autarkiegrad heute in Prozent. Die Berechnung erfolgt nur wenn Consumption Sensor konfiguriert ist. Der State Class ist Measurement.

Expected Daily Production zeigt die erwartete Tagesproduktion die am Morgen prognostiziert wurde in kWh. Der Wert wird persistent gespeichert. Die Aktualisierung erfolgt täglich um 6 Uhr.

Next Hour Prediction ist optional und muss aktiviert werden. Der Sensor zeigt die Prognose für die nächste Stunde in kW. Die Berechnung verwendet ML-Model oder Profile. Der State Class ist Measurement.

Diagnostic Sensoren

Diese Sensoren sind verfügbar wenn Diagnostic Mode aktiviert ist.

Diagnostic Status zeigt den Gesamtstatus der Integration. Mögliche Werte sind Ready, Learning, Initializing, Degraded, Error. Der Sensor zeigt kritische Probleme sofort.

Solar Forecast Accuracy zeigt die Model Accuracy in Prozent. Die Berechnung erfolgt aus R-squared Score des ML-Models. Der Wert wird nach jedem Training aktualisiert.

Yesterday Forecast Deviation zeigt die Abweichung der gestrigen Prognose in kWh und Prozent. Der Wert wird täglich um 21 Uhr berechnet. Positive Werte bedeuten Überproduktion, negative Werte Unterproduktion.

Last Coordinator Update zeigt den Timestamp des letzten erfolgreichen Updates. Das Format ist ISO 8601. Der Sensor hilft bei Debugging von Update-Problemen.

Update Age zeigt wie lange das letzte Update her ist in Minuten. Der Sensor warnt bei zu alten Daten. Threshold ist update interval mal zwei.

Last ML Training zeigt den Timestamp des letzten erfolgreichen Trainings. Das Format ist ISO 8601. Der Wert wird persistent gespeichert.

Next Scheduled Update zeigt wann das nächste automatische Update geplant ist. Die Berechnung berücksichtigt scheduled tasks und coordinator interval. Das Format ist ISO 8601.

ML Service Status zeigt den Status des ML Predictors. Mögliche Werte sind Ready, Training, Degraded, Unavailable, Error. Der Sensor zeigt Model State und Health.

ML Metrics zeigt detaillierte ML-Metriken. Die Attribute umfassen Model State, Training Samples Count, Feature Count, Last Training Duration, Performance Metrics. Der Sensor ist nur bei aktiviertem Learning verfügbar.

Coordinator Health zeigt den Health Status des Coordinators. Die Attribute umfassen Services Initialized, ML Ready, Weather Service Status, Error Count, Last Update Time. Der Sensor aggregiert alle kritischen Status-Informationen.

Data Files Status zeigt den Status aller persistenten Dateien. Die Attribute umfassen File Sizes, Last Modified Timestamps, Backup Count, Disk Space. Der Sensor hilft bei Speicherproblemen.

State Sensoren

Diese Sensoren spiegeln externe Sensor-Werte wider.

External Temperature zeigt Temperatur vom konfigurierten Sensor oder Weather Entity. Die Einheit ist Grad Celsius. Der State Class ist Measurement.

External Humidity zeigt relative Luftfeuchtigkeit in Prozent. Die Quelle ist Humidity Sensor oder Weather Entity. Der State Class ist Measurement.

External Wind Speed zeigt Windgeschwindigkeit. Die Einheit ist m/s oder km/h je nach Sensor. Der State Class ist Measurement.

External Rain zeigt Niederschlagsmenge in mm. Die Quelle ist Rain Sensor falls konfiguriert. Der State Class ist Measurement.

External UV Index zeigt UV-Index. Der Wertebereich ist 0 bis 11 plus. Die Quelle ist UV Sensor falls konfiguriert.

External Lux zeigt Lichtintensität in Lux. Die Quelle ist Lux Sensor falls konfiguriert. Der State Class ist Measurement.

Power Sensor State zeigt den aktuellen Wert des Power Sensors in Watt. Der Sensor dient zur Überwachung der Datenqualität. Der State Class ist Measurement.

Yield Sensor State zeigt den aktuellen Wert des Yield Sensors in kWh. Der Sensor zeigt die Tagesproduktion. Der State Class ist Total Increasing.

---

8. Buttons und Services

Buttons

Die Integration stellt zwei Buttons zur Verfügung.

Manual Forecast Button mit Icon mdi:refresh-circle erzwingt sofortiges Update mit frischen Wetterdaten, aktualisiert alle Prognosen, berechnet Expected Daily Production neu und triggert Sensor Updates. Der Button ist immer verfügbar.

Manual Learning Button mit Icon mdi:brain startet manuelles ML-Training, nutzt alle verfügbaren Samples, berechnet neue Model Accuracy, persistiert Learned Weights und aktualisiert ML Service Status. Der Button ist nur verfügbar wenn Learning aktiviert ist.

Services

Die Integration registriert zwei Home Assistant Services.

solar_forecast_ml.force_retrain erzwingt vollständiges Retraining des ML-Models. Das Training nutzt alle verfügbaren historischen Daten, ignoriert Accuracy Threshold, berechnet neue Weights und Feature Importance und persistiert Model State. Der Service hat keine Parameter. Der Service ist nur verfügbar wenn Learning aktiviert ist.

solar_forecast_ml.reset_model setzt das ML-Model zurück auf Anfangszustand. Die Aktion löscht alle Learned Weights, setzt Model State auf UNINITIALIZED, behält aber Trainingsdaten und erzwingt Retraining beim nächsten Cycle. Die Warnung lautet alle gelernten Parameter gehen verloren. Der Service hat keine Parameter.

---

9. Datenmanagement und Persistenz

Dateistruktur

Alle Daten werden in folgendem Verzeichnis gespeichert: /config/solar_forecast_ml/

Die persistenten Dateien sind wie folgt organisiert.

prediction_history.json speichert historische Tagesprognosen und tatsächliche Werte. Die Struktur umfasst date, predicted_value, actual_value, deviation_kwh, deviation_percent. Das Maximum beträgt 365 Einträge. Das Cleanup erfolgt automatisch.

learned_weights.json speichert die trainierten Model-Gewichte. Die Struktur umfasst weights array, intercept, feature_names, model_metadata, training_timestamp. Die Größe beträgt typisch 5 bis 20 KB. Das Update erfolgt nach jedem Training.

hourly_profile.json speichert historische Produktionsprofile. Die Struktur umfasst hourly_averages für jede Stunde 0 bis 23, monthly_patterns und seasonal_adjustments. Die Größe beträgt typisch 10 bis 30 KB. Das Update erfolgt täglich.

model_state.json speichert den aktuellen Model-Status. Die Struktur umfasst current_state, last_training_time, model_accuracy, training_samples_count, feature_importance. Die Größe beträgt typisch 2 bis 5 KB. Das Update erfolgt nach Status-Änderungen.

hourly_samples.json speichert stündliche Produktionsdaten für Training. Die Struktur umfasst timestamp, power_w, weather_data, daily_production, external_sensors. Das Maximum beträgt 1440 Samples etwa 60 Tage. Das Cleanup erfolgt automatisch.

weather_cache.json cached Wetterdaten um API-Calls zu reduzieren. Die Struktur umfasst cached_forecasts, timestamp, source_entity. Das Timeout beträgt 1 Stunde. Das Cleanup erfolgt automatisch.

versinfo.json speichert Versions- und Migrations-Informationen. Die Struktur umfasst current_version, last_migration, feature_flags. Die Größe beträgt kleiner als 1 KB. Das Update erfolgt bei Integration Updates.

Backup-System

Das automatische Backup-System funktioniert wie folgt.

Das Backup wird bei kritischen Operationen erstellt: vor Model Reset, vor Migration, bei Version Updates. Das Backup Location ist /config/solar_forecast_ml/backups/backup_TIMESTAMP/. Die Retention beträgt 30 Tage oder 10 Backups maximal. Das Cleanup erfolgt automatisch bei Überschreitung der Limits.

Atomic Write Operations

Alle Schreiboperationen sind atomar. Der Prozess läuft wie folgt ab. Erstens Schreiben in Temporary File mit unique Name. Zweitens Validation der geschriebenen Daten. Drittens Atomic Move über shutil.move zur Zieldatei. Viertens Cleanup des Temp Files bei Fehler. Das Ergebnis ist keine korrupten Dateien durch Crashes und konsistente Daten auch bei Stromausfall.

Datenintegrität

Die Datenintegrität wird sichergestellt durch mehrere Mechanismen.

JSON Schema Validation erfolgt bei jedem Load. Type Checking erfolgt mit TypedDataAdapter. Range Validation erfolgt für physikalische Größen. Consistency Checks erfolgen zwischen verbundenen Daten. Error Recovery nutzt Default Values bei korrupten Dateien. Automatic Repair versucht Daten zu reparieren wo möglich.

---

10. Zeitzonenhandling

Kritisches Zeitzonenverhalten

Die gesamte Integration arbeitet zwingend in Local Time. Alle Berechnungen erfolgen in der System-Zeitzone von Home Assistant. Alle persistierten Datetime-Objekte werden in Local Time gespeichert. Alle Recorder-Daten werden von UTC in Local Time umgerechnet.

SafeDateTimeUtil Helper

Die Klasse SafeDateTimeUtil stellt zentrale Zeitfunktionen bereit.

ensure_local konvertiert jeden datetime zu Local Time und entfernt naive datetime. now liefert current time in Local Time. today liefert today start 00:00:00 in Local Time. parse_datetime parst ISO strings mit automatischer Timezone-Konvertierung.

Recorder Integration

Bei der Arbeit mit Home Assistant Recorder ist folgendes zu beachten.

Recorder speichert alle Daten in UTC. Die Integration muss UTC zu Local konvertieren bei allen Recorder-Queries. Die Funktion dt_util.ensure_local wird systematisch angewendet auf alle recorder.history Ergebnisse. Die History Start und End Times werden in Local Time angegeben aber intern als UTC verarbeitet.

Zeitbasierte Berechnungen

Alle zeitbasierten Operationen nutzen Local Time.

Hour of Day für stündliche Sampling und Profile verwendet Local Time. Day Boundaries für tägliche Resets und Aggregation verwendet Mitternacht Local Time. Scheduled Tasks für Morning Update und Evening Verification verwendet Local Time Hours. Sunrise und Sunset Calculations verwendet Local Time mit Offset.

Common Pitfalls

Häufige Fehlerquellen werden vermieden.

Nie direkt datetime.utcnow verwenden, immer dt_util.now. Nie datetime.now ohne Timezone verwenden. Nie naive datetime Objekte vergleichen mit aware datetime. Immer ensure_local auf Recorder-Daten anwenden. Immer Local Time für User-facing Timestamps verwenden.

---

11. Fehlerbehandlung und Stabilität

Error Handling Service

Der zentrale ErrorHandlingService verwaltet alle Fehler.

Das Circuit Breaker Pattern verhindert Cascade Failures. Der Failure Threshold beträgt 5 aufeinanderfolgende Fehler. Der Recovery Timeout beträgt 60 Sekunden. Der Success Threshold beträgt 3 erfolgreiche Calls zur Erholung. Das Ergebnis ist automatische Deaktivierung bei wiederholten Fehlern und automatische Recovery wenn Service wieder stabil.

Exception Hierarchy

Die Integration definiert eigene Exception-Typen.

SolarForecastMLException ist die Basis-Exception für alle Integration-Fehler mit Error Severity Low, Medium, High, Critical und Context Information als Dict.

WeatherAPIException wird genutzt für Wetterdaten-Abruf-Fehler mit Weather Entity spezifischen Infos und Retry-Logik.

MLModelException wird genutzt für ML-Training und Prediction-Fehler mit Model State Informationen und Feature-bezogenen Details.

DataIntegrityException wird genutzt für Datei-IO und Persistenz-Fehler mit File Path und Operation Details und Backup-Informationen.

Retry-Mechanismen

Verschiedene Retry-Strategien werden eingesetzt.

Weather Data Fetch hat 3 Versuche mit exponential backoff, 5 Sekunden initial delay und Fallback zu cached Data.

ML Training hat 2 Versuche bei Resource-Fehlern, kein Retry bei Daten-Problemen und Fallback zu Previous Model.

Data Persistence hat 3 Versuche mit 1 Sekunde delay, atomare Operationen verhindern Korruption und Fallback zu Default Values.

Graceful Degradation

Das System degradiert schrittweise bei Problemen.

Bei ML-Fehler erfolgt Fallback auf regelbasierte Prognose mit Warnung im Log und Sensor zeigt Degraded State.

Bei Weather-Fehler erfolgt Verwendung von Cached Data mit reduzierten Features und Notification an User.

Bei Data-Fehler erfolgt Verwendung von Default Values mit Versuch der Daten-Reparatur und Fortsetzung der Operation.

Bei Dependency-Fehler erfolgt Deaktivierung betroffener Features mit klare User-Kommunikation und Core-Funktionen bleiben verfügbar.

Logging

Die Integration nutzt strukturiertes Logging auf verschiedenen Levels.

DEBUG für detaillierte Ablauf-Informationen, Daten-Transformationen und Performance-Metriken.

INFO für normale Operationen, erfolgreiche Updates und Status-Änderungen.

WARNING für nicht-kritische Probleme, Fallback-Aktivierungen und veraltete Konfigurationen.

ERROR für schwerwiegende Fehler, fehlgeschlagene Operationen und Exception-Details.

CRITICAL für System-kritische Fehler, Datenverlust-Risiken und erforderliche User-Intervention.

Health Monitoring

Das System überwacht kontinuierlich seine Gesundheit.

Coordinator Health Check prüft Services Initialized, ML Predictor Ready, Weather Service Available, Last Update Success und Error Count.

ML Health Check prüft Model State, Training Samples Count, Model Accuracy, Feature Availability und Last Training Time.

Data Health Check prüft File Integrity, Backup Status, Disk Space und Last Write Success.

---

12. Besonderheiten und Einschränkungen

Voraussetzungen

Die Integration hat spezifische Anforderungen.

Home Assistant Core Version 2024.1 oder höher. Python Dependencies sind numpy größer gleich 1.21.0 und aiofiles größer gleich 23.0.0. Recorder Component für historische Daten und ML-Training. Weather Entity mit Forecast-Fähigkeit. Power Sensor mit kontinuierlichen Updates ideal jede Sekunde bis Minute. Solar Yield Sensor mit täglichem Reset um Mitternacht.

Hardware-Anforderungen

Die Ressourcen-Nutzung ist moderat.

CPU-Last beträgt während ML-Training 10 bis 30 Prozent für 1 bis 5 Minuten, normal Betrieb unter 5 Prozent. RAM-Nutzung beträgt mit aktivem ML etwa 50 bis 100 MB, ohne ML etwa 20 bis 50 MB. Disk Space beträgt für Daten etwa 5 bis 20 MB, für Backups etwa 20 bis 50 MB. Das System ist optimiert für Raspberry Pi 4 und vergleichbare Systeme.

Bekannte Einschränkungen

Mehrere Einschränkungen sind zu beachten.

Nur eine Wetter-Entity wird primär genutzt ohne automatischen Fallback zu alternativen Quellen. Keine Unterstützung für Multiple Arrays oder Split Systems, nur eine Gesamtkapazität. Kein Real-Time Prediction außer stündlich zur vollen Stunde bei aktiviertem Hourly Mode. Keine Integration mit externen Forecast APIs wie Solcast oder Forecast.Solar. Keine automatische Anlagenoptimierung oder Empfehlungen.

Performance-Optimierungen

Die Integration ist auf Effizienz optimiert.

Lazy Loading von numpy erfolgt nur wenn tatsächlich benötigt. Caching von Weather Data mit 1 Stunde Timeout. Atomic Operations in ThreadPoolExecutor für blocking IO. Asyncio für alle Network und File Operations. Minimal Sensor Updates nur bei tatsächlichen Änderungen.

Skalierbarkeit

Die Grenzen der Skalierbarkeit sind wie folgt.

Maximum Prediction History beträgt 365 Tage automatisches Cleanup. Maximum Hourly Samples beträgt 1440 etwa 60 Tage. Maximum Backup Files beträgt 10 automatisches Cleanup. Maximum Concurrent Operations beträgt 3 zur Vermeidung von Resource Starvation.

Zukunftsperspektive

Geplante Erweiterungen umfassen mehrere Bereiche.

Multi-Array Support für Split Systems mit verschiedenen Ausrichtungen. Enhanced ML Models mit LSTM oder Transformer Architectures. Real-Time Predictions mit minutengenauer Auflösung. External API Integration für Forecast Services. Battery Storage Integration für Optimierung. Weather Forecast Comparison mit verschiedenen Quellen. Advanced Visualizations mit benutzerdefinierten Charts.

Migration und Updates

Bei Integration-Updates ist folgendes zu beachten.

Automatische Datenmigration bei Format-Änderungen. Backup vor Major Updates empfohlen. Kompatibilität mit alten Daten-Formaten für 2 Major Versions. Breaking Changes werden im Release Notes dokumentiert. Downgrade ist möglich mit Backup-Wiederherstellung.

Troubleshooting

Häufige Probleme und Lösungen sind wie folgt.

Bei fehlenden Prognosen prüfe Weather Entity Verfügbarkeit, checke Coordinator Health Sensor und schaue in Home Assistant Logs nach Fehlern.

Bei ML-Training schlägt fehl prüfe mindestens 50 Datenpunkte vorhanden, checke numpy Installation und schaue ML Service Status Sensor an.

Bei inakkuraten Prognosen warte auf mehr Trainingsdaten mindestens 200 Samples, prüfe Power Entity Genauigkeit und checke Weather Entity Qualität.

Bei hoher CPU-Last reduziere Update Interval, deaktiviere Hourly Mode falls nicht benötigt und checke auf andere ressourcenintensive Integrationen.

Bei Speicherproblemen führe manuelles Backup Cleanup durch, reduziere Prediction History Limit und prüfe verfügbaren Disk Space.

---

13. Nutzungsrechte und Einschränkungen

Lizenzierung

Diese Software steht unter der GNU Affero General Public License Version 3 wie von der Free Software Foundation veröffentlicht. Sie dürfen die Software verbreiten und modifizieren unter den Bedingungen dieser Lizenz. Der vollständige Lizenztext ist verfügbar unter https://www.gnu.org/licenses/agpl-3.0.html

Private und nicht-kommerzielle Nutzung

Die Integration ist ausdrücklich für den privaten Gebrauch in Home Assistant Installationen konzipiert. Privatpersonen dürfen die Software frei nutzen für ihre persönlichen Solaranlagen. Die Installation und Verwendung in privaten Haushalten ist kostenlos und ohne weitere Genehmigung gestattet. Modifikationen für den Eigenbedarf sind erlaubt und ausdrücklich erwünscht. Das Teilen von Erfahrungen und Verbesserungsvorschlägen in der Community ist willkommen.

Strikte Untersagung kommerzieller Nutzung ohne Genehmigung

Der Autor Zara-Toorox untersagt hiermit ausdrücklich und in aller Deutlichkeit jegliche kommerzielle Nutzung dieser Integration ohne vorherige schriftliche Genehmigung.

Insbesondere ist ohne explizite schriftliche Einwilligung des Autors strengstens untersagt die Verwendung in kommerziellen Solarpark-Betrieben oder Anlagen mit gewerblicher Stromproduktion. Der Verkauf oder die Lizenzierung der Software oder Teilen davon an Dritte ist verboten. Die Integration der Software in kommerzielle Produkte oder Dienstleistungen ist untersagt. Die Bereitstellung als kostenpflichtiger Service oder Software-as-a-Service ist verboten. Die Nutzung zur Generierung von Einnahmen durch Werbung oder Affiliate-Marketing ist untersagt. Die Verwendung in Beratungsdienstleistungen oder als Teil kostenpflichtiger Installationen ist verboten. Der Weiterverkauf von aus dieser Software gewonnenen Daten oder Prognosen ist untersagt. Die Nutzung in Forschungsprojekten mit kommerziellem Hintergrund ohne Genehmigung ist verboten.

Kommerzielle Lizenzierung

Unternehmen oder Organisationen die die Software kommerziell nutzen möchten müssen eine kommerzielle Lizenz beim Autor beantragen. Für Anfragen zur kommerziellen Nutzung kontaktieren Sie bitte den Autor über die GitHub Repository Seite https://github.com/Zara-Toorox/ha-solar-forecast-ml. Individuelle Lizenzvereinbarungen können nach Prüfung des Einzelfalls geschlossen werden. Die Konditionen für kommerzielle Lizenzen werden fallspezifisch verhandelt.

Rechtliche Konsequenzen bei Missachtung

Verstöße gegen diese Nutzungsbedingungen werden strafrechtlich und zivilrechtlich verfolgt. Bei unerlaubter kommerzieller Nutzung behält sich der Autor vor Schadensersatzforderungen in Höhe der entgangenen Lizenzgebühren plus Gewinnabschöpfung geltend zu machen. Einstweilige Verfügungen zur sofortigen Unterlassung werden bei Bedarf beantragt. Zusätzlich können Anwalts- und Gerichtskosten geltend gemacht werden. Eine Verletzung

der AGPL-3.0 Lizenzbestimmungen führt zum automatischen Erlöschen aller Nutzungsrechte.

Haftungsausschluss

Die Software wird bereitgestellt wie sie ist ohne jegliche Gewährleistung ausdrücklich oder impliziert. Der Autor haftet nicht für Schäden die aus der Nutzung oder Nichtnutzbarkeit der Software entstehen. Dies umfasst direkte, indirekte, zufällige oder Folgeschäden. Insbesondere wird keine Haftung übernommen für finanzielle Verluste durch ungenaue Prognosen, Datenverlust oder Systemausfälle oder Schäden an Hard- oder Software. Der Nutzer trägt die volle Verantwortung für die Verwendung der Software und deren Integration in sein System.

Attribution und Nennung

Bei jeglicher Verwendung Weiterverbreitung oder Modifikation der Software muss die Urheberschaft von Zara-Toorox klar erkennbar bleiben. Copyright-Hinweise und Lizenzinformationen dürfen nicht entfernt oder verändert werden. Bei Veröffentlichung von Modifikationen muss auf die ursprüngliche Quelle verwiesen werden. Die AGPL-3.0 Lizenz erfordert dass auch modifizierte Versionen unter derselben Lizenz veröffentlicht werden müssen.

Beiträge und Community

Beiträge zur Verbesserung der Software sind willkommen und geschätzt. Pull Requests auf GitHub können eingereicht werden und werden vom Autor geprüft. Durch das Einreichen von Code stimmen Sie zu dass Ihr Beitrag unter der AGPL-3.0 Lizenz veröffentlicht wird. Bug Reports und Feature Requests können über das GitHub Issue Tracking System eingereicht werden. Community-Support erfolgt auf freiwilliger Basis ohne Garantie.

Datenschutz

Die Software verarbeitet ausschließlich lokale Daten in Ihrer Home Assistant Installation. Es werden keine Daten an externe Server oder den Autor übermittelt. Alle Produktionsdaten und Prognosen verbleiben in Ihrem System. Die Integration kommuniziert nur mit Ihren konfigurierten Home Assistant Entitäten und Weather Services. Der Autor hat keinen Zugriff auf Ihre Daten oder Installationen.

Änderungsvorbehalt

Der Autor behält sich das Recht vor diese Nutzungsbedingungen jederzeit zu ändern. Änderungen werden über das GitHub Repository kommuniziert. Die Nutzung neuerer Versionen impliziert die Akzeptanz der dann gültigen Bedingungen. Bestehende Installationen bleiben von Änderungen unberührt sofern keine Updates eingespielt werden.

Schlusswort

Die Solar Forecast ML Integration ist eine komplexe aber robuste Lösung für präzise Solarprognosen. Durch die Kombination aus regelbasierten Ansätzen und maschinellem Lernen erreicht sie hohe Genauigkeit bei gleichzeitiger Fehlertoleranz. Die modulare Architektur ermöglicht einfache Wartung und Erweiterung. Die ausführliche Fehlerbehandlung garantiert stabilen Betrieb auch unter ungünstigen Bedingungen.

Was diese Integration von klassischen Forecast-Systemen grundlegend unterscheidet ist ihre Fähigkeit zur kontinuierlichen Anpassung und Verbesserung. Während kommerzielle Systeme oft statisch bleiben und allgemeine Modelle verwenden lernt diese Integration die spezifischen Charakteristika Ihrer individuellen Anlage. Sie berücksichtigt lokale Mikroklima-Effekte, Verschattungsmuster und andere standortspezifische Faktoren automatisch ohne dass Sie diese manuell konfigurieren müssen.

Die intelligente Strategie-Auswahl mit mehrfacher Absicherung durch Ensemble-Methoden und automatischem Fallback garantiert dass Sie immer eine Prognose erhalten selbst wenn einzelne Komponenten ausfallen oder noch keine ausreichenden Trainingsdaten vorhanden sind. Das System degradiert gracefully und erholt sich automatisch sobald die Bedingungen wieder optimal sind.

Bei korrekter Konfiguration und ausreichenden Trainingsdaten liefert die Integration zuverlässige Prognosen die zur Optimierung des Eigenverbrauchs und zur Planung des Energieeinsatzes genutzt werden können. Die Integration lernt kontinuierlich und passt sich an die spezifischen Gegebenheiten der Anlage an wobei die Genauigkeit typischerweise nach zwei bis vier Wochen ein hohes Niveau erreicht und sich danach weiter stabilisiert.

Für optimale Ergebnisse sollte die Integration mindestens zwei Wochen Zeit zum Sammeln von Daten und Training erhalten. Danach verbessert sich die Genauigkeit kontinuierlich mit zunehmender Datenmenge. Nach etwa einem Monat hat das System genug Daten über verschiedene Wetterbedingungen und Tageszeiten gesammelt um sehr präzise Vorhersagen zu treffen. Nach einem kompletten Jahreszyklus kennt das System alle saisonalen Muster und erreicht seine maximale Vorhersagegenauigkeit.

Diese Integration wurde mit viel Aufwand und Liebe zum Detail entwickelt um der Open Source Community ein leistungsfähiges Werkzeug zur Verfügung zu stellen. Sie dürfen sie frei für private Zwecke nutzen und zur Verbesserung beitragen. Bitte respektieren Sie jedoch die Nutzungsbedingungen insbesondere hinsichtlich der kommerziellen Nutzung. Nur so kann sichergestellt werden dass die Entwicklung und Pflage der Software langfristig fortgeführt werden kann.

Der Autor freut sich über Feedback Bug Reports und Verbesserungsvorschläge über das GitHub Repository. Gemeinsam können wir diese Integration weiter verbessern und zu einem noch besseren Werkzeug für die Solar-Community machen.

Zara